SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I Roshini B T/192524064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**Roshini B T**

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;
   for(i = 0; i <= n; i++) {
      sum = sum + arr[i];
   }
   return sum;
}

int main() {
   int arr[5] = {1,2,3,4,5}
   int total;

   total = sumArray(arr);

   if(total == 15) {
      printf("Correct Sum\n")
   }

   int a = 10, b = 0;
   int c = a / b;

   int *p;
   *p = 20;

   return 0;
}
```

Error Analysis

(a) Error Identification

The given program contains errors at different stages of compilation. Each error is explained below in a simple and humanised manner. The original task specifically asks for the line number and nature of each error.

Line	Error	Type	Explanation
5	i <= n	Logical / Runtime	The loop accesses arr[n], which is outside the valid array range. It should use i < n.
12	Missing ;	Syntax	The array declaration is not terminated with a semicolon.
15	sumArray(arr)	Semantic	The function requires two arguments, arr and n, but only one is supplied.
18	Missing ;	Syntax	The printf() statement needs a semicolon.
22	a / b where b = 0	Runtime	Division by zero causes an execution-time error.
25	*p = 20	Runtime	p has not been initialized to a valid memory address, so dereferencing it is unsafe.

(b) Classification of Errors

The errors can be grouped into the compiler/error categories specified in the assessment.

Lexical Errors:

No major lexical error is present in the given Q1 program. All keywords, identifiers, and operators are written using valid tokens.

Syntax Errors:

- Line 58 – Missing semicolon after array initialization.
- Line 64 – Missing semicolon after printf().

These errors violate the grammatical rules of C.

Semantic Error:

- Line 61 – Incorrect number of arguments passed to sumArray().

The statement may look syntactically correct, but it does not match the function definition.

Logical Error:

- Line 51 – i <= n should be i < n.

The program's intention is to process five array elements, but the current condition allows one extra access.

Runtime Errors:

- Line 68 – Division by zero.
- Line 71 – Dereferencing an uninitialized pointer.

These problems become critical when the program is executed.

(c) Compilation Behaviour

The compiler can detect syntax and certain semantic errors before the program runs. For example, the missing semicolons and incorrect function arguments can generate compilation errors.

However, problems such as division by zero and invalid pointer access may only become apparent during execution. The loop-bound error can also cause an invalid array access at runtime.

Early errors can prevent later errors from being detected. For example, if the compiler encounters a missing semicolon, it may not correctly interpret the statements that follow it. Therefore, the programmer normally needs to fix compilation errors first before checking runtime behaviour.

(d) Corrected Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;

   for (i = 0; i < n; i++) {
      sum = sum + arr[i];
   }

   return sum;
}

int main() {
   int arr[5] = {1, 2, 3, 4, 5};
   int total;

   total = sumArray(arr, 5);

   if (total == 15) {
      printf("Correct Sum\n");
   }

   int a = 10, b = 1;
   int c = a / b;

   int value = 20;
   int *p = &value;

   *p = 20;

   printf("Division result: %d\n", c);

   return 0;
}
```

The corrected program now has proper syntax, passes the required function arguments, stays within the array boundary, avoids division by zero by using a non-zero divisor (b = 1), and initializes the pointer before using it. This makes the example safe to execute.

(e) Impact Analysis

1. Improper pointer usage:

The statement *p = 20 is unsafe because p does not point to a valid memory location. It may result in a segmentation fault or unexpected memory corruption.

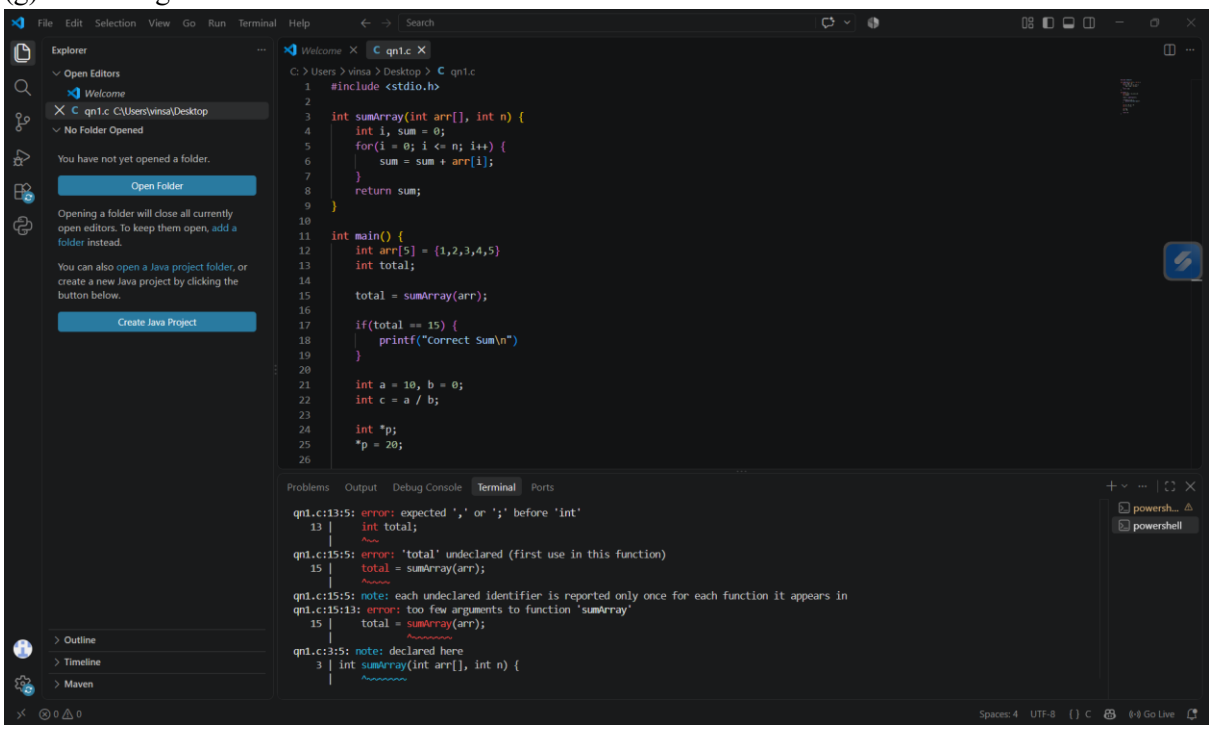
2. Division by zero:
The statement a / b with b = 0 is mathematically undefined. During execution, it can terminate the program or generate a runtime exception depending on the system.
3. Array boundary violation:
Using i <= n causes the program to access one position beyond the valid array range. This may produce incorrect results or corrupt memory.
4. Incorrect function call:
Calling sumArray(arr) without the second argument means the function does not receive all the information required to perform its task.
These errors can therefore lead to compilation failure, incorrect output, program crashes, or unsafe memory access.
(f) Advanced Compiler Perspective
A compiler can improve error handling by providing messages that clearly identify:

- The exact line and position of the error.
- The expected syntax.
- A simple explanation of the problem.
- A possible correction.

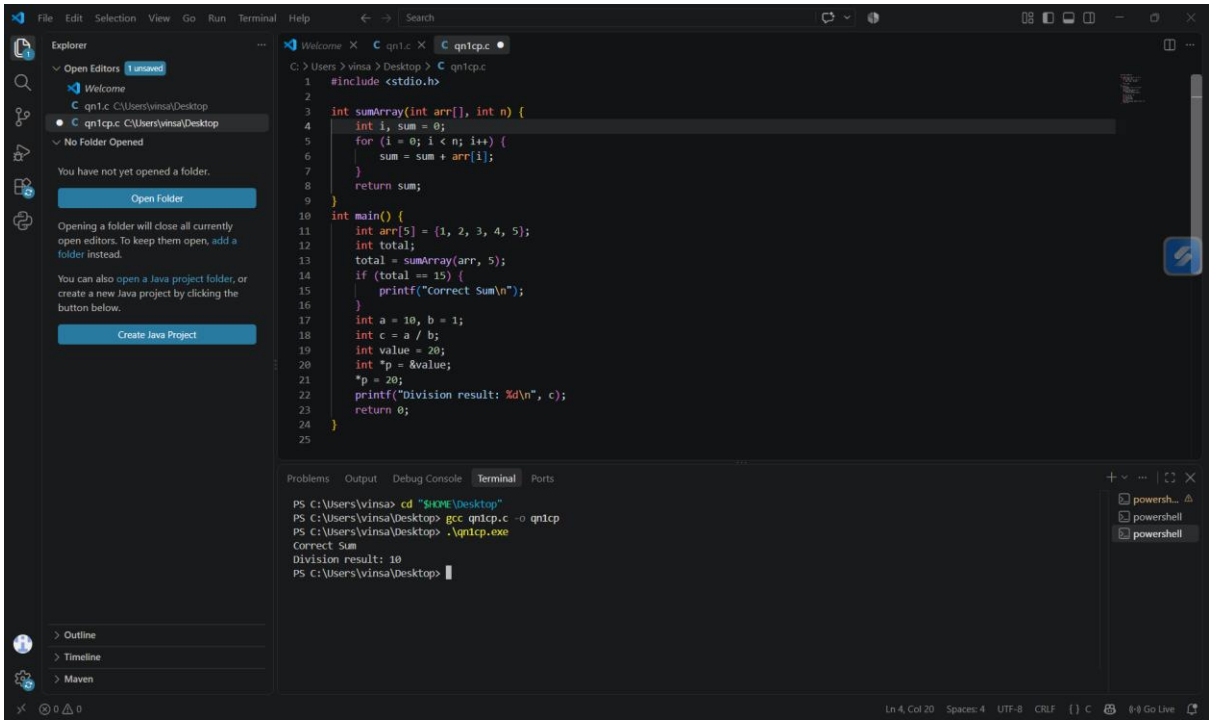
For example, instead of simply reporting **"expected ';'"**, the compiler could say:
Line 58: A semicolon is missing after the array initialization. Add ; at the end of the statement.
Two useful compiler-design enhancements are:

1. **Static analysis:** Detect possible division by zero, uninitialized pointers, and array-bound violations before execution.
2. **Intelligent debugging support:** Provide source-line information, possible causes, and suggested corrections.

(g) Error Program Screenshot



(h)Corrected Program Screenshot



Q2. The following C program contains multiple lexical errors introduced intentionally. These errors occur due to invalid tokens, incorrect identifiers, malformed constants, and illegal symbols, which are typically detected during the lexical analysis phase of a compiler. Carefully examine the program and answer the questions given below.

```
#include <stdio.h>

int main() {
    int 1num = 100;
    float rate@ = 25.5;
    char ch = 'abc';
    int total = 50$20;
    int num#1 = 30;
    int %count = 40;

    printf("Welcome to Compiler Design);

    int x = 09;
    int y = 0xZ12;
    int z = 12.3.4;

    char str1[] = "Hello;
    char str2[] = "World";

    int _valid = 10;
    int invalid-id = 20;

    float value = 5..6;
    int data = 10@5;

    char c = 'xy';
```

```
    int 3value = 60;

    printf("Value: %d\n", total);

    return 0;
}
```

Q2. Lexical Error Analysis

Q2 specifically focuses on lexical errors caused by invalid tokens, identifiers, constants, literals, and symbols.

(a) Error Identification

Line	Invalid Code	Error	Reason
4	1num	Invalid identifier	An identifier cannot begin with a digit.
5	rate@	Invalid identifier	@ is not permitted inside a C identifier.
6	'abc'	Invalid character literal	A character literal should contain one character.
7	50$20	Invalid token	$ is not a valid C operator or separator here.
8	num#1	Invalid identifier	# cannot appear inside an identifier.
9	%count	Invalid identifier	% is an operator, not a valid identifier character.
11	"Welcome to Compiler Design)	Unterminated string	The closing quotation mark is missing.
13	09	Invalid numeric constant	The value is interpreted as an octal-style constant, where 9 is invalid.
14	0xZ12	Invalid hexadecimal constant	Z is not a hexadecimal digit.
15	12.3.4	Invalid floating constant	A floating-point number cannot contain two decimal points.
17	"Hello	Unterminated string	The closing quotation mark is missing.
19	invalid-id	Invalid identifier	- is interpreted as the subtraction operator.
20	5..6	Invalid floating constant	Two decimal points are used.
21	10@5	Invalid token	@ is not valid in this expression.
23	'xy'	Invalid character literal	A character literal should contain one character.
24	3value	Invalid identifier	An identifier cannot begin with a digit.

These errors directly relate to the task's requirement to identify invalid tokens and line numbers.

(b) Error Classification

The errors violate different token-formation rules.

Identifier Rules

Valid C identifiers generally:

- Start with a letter or _.
- May contain letters, digits, and _.
- Cannot contain symbols such as @, #, %, or -.

Therefore:

1num → Invalid
rate@ → Invalid
num#1 → Invalid

%count → Invalid
invalid-id → Invalid
3value → Invalid

Constant Formation

The following constants are malformed:
09
0xZ12
12.3.4
5..6

String/Character Literal Rules

Invalid examples:
'abc'
"Welcome to Compiler Design)
"Hello
'xy'

A standard single-character constant should contain one character. Multi-character character constants such as 'abc' and 'xy' are not suitable single-character values and may produce a compiler diagnostic; replace them with 'a' and 'x'.

Operator/Symbol Rules

Symbols such as $, @, #, and % cannot simply be inserted into identifiers or expressions where they do not have a valid lexical meaning. These violations correspond to the token-formation categories specified in the assessment.

(c) Token Correction

Invalid Token	Correct Token
1num	num1
rate@	rate
'abc'	'a'
50$20	50 + 20
num#1	num1
%count	count
"Welcome to Compiler Design)	"Welcome to Compiler Design"
09	9
0xZ12	0xA12
12.3.4	12.34
"Hello	"Hello"
invalid-id	invalid_id
5..6	5.6
10@5	10 + 5
'xy'	'x'
3value	value3

The corrections follow the assessment's example of converting an invalid identifier such as 2value into a valid form such as value2.

(d) Compiler Behaviour Analysis

The lexical analyzer, or scanner, reads the source program character by character and groups characters into tokens.

When it encounters an invalid sequence, it reports a lexical error. Depending on the compiler's error-recovery mechanism, it may either:

1. Stop processing immediately, or
2. Report the error and continue scanning from a suitable point.

Modern compilers generally attempt error recovery so that multiple errors can be reported in one compilation attempt. This allows the programmer to correct several problems instead of compiling the program repeatedly after every individual error.

Corrected Program:

```
#include <stdio.h>

int main() {
    int num = 100;
    float rate = 25.5;
    char ch = 'a';
    int total = 50 + 20;
    int num1 = 30;
    int count = 40;

    printf("Welcome to Compiler Design\n");

    int x = 9;
    int y = 0xA12;
    float z = 12.34;

    char str1[] = "Hello";
    char str2[] = "World";

    int _valid = 10;
    int invalid_id = 20;

    float value = 5.6;
    int data = 10 + 5;

    char c = 'x';
    int value3 = 60;

    printf("Value: %d\n", total);

    return 0;
}
```

(e) Advanced Thinking

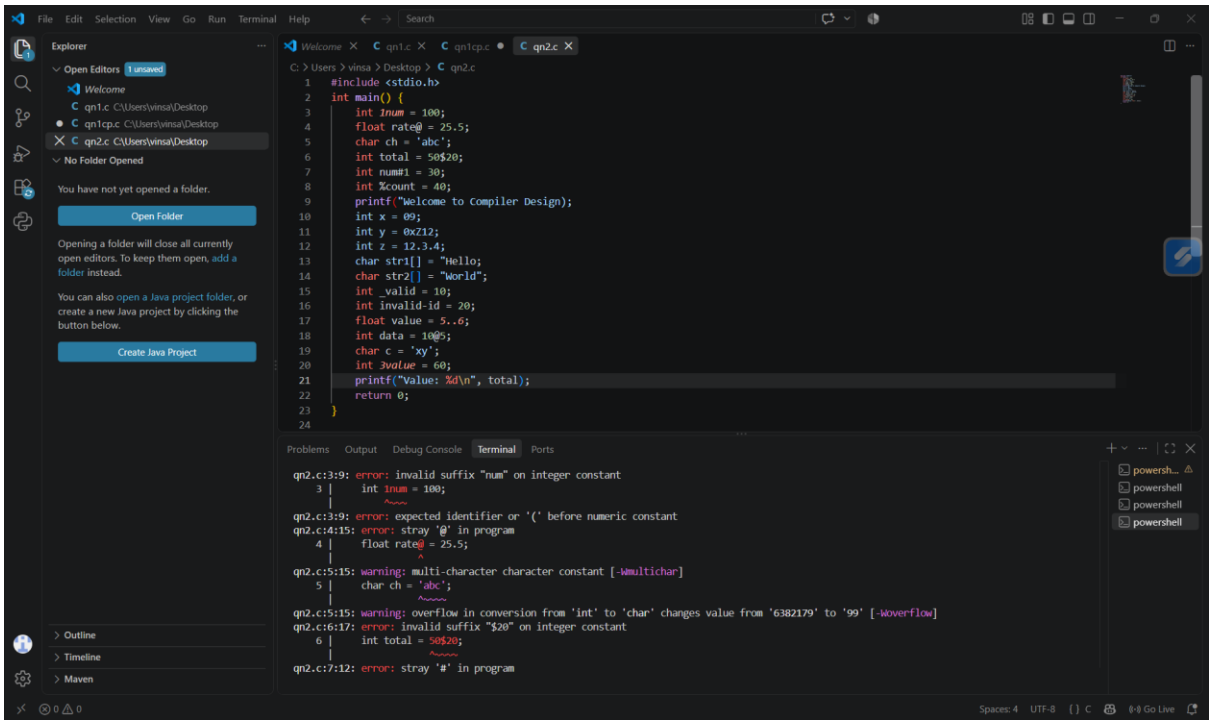
DFA Idea for Detecting Invalid Identifiers

A simple DFA can be designed as follows:

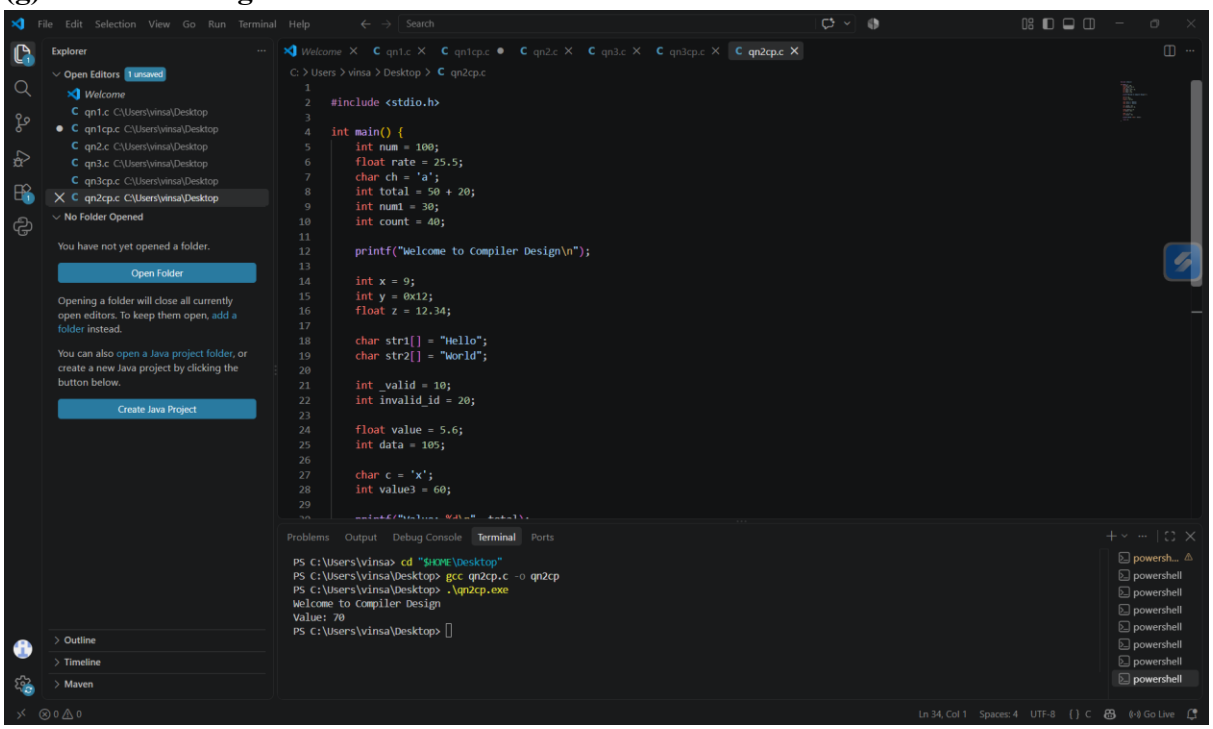
```
START
  |
  | Letter / _
  ↓
VALID IDENTIFIER
  |
  | Letter / Digit / _
  ↓
VALID IDENTIFIER
```

START
|
| Digit
↓
INVALID IDENTIFIER
If the first character is a digit, the scanner can immediately identify that the sequence cannot be a valid C identifier.

(f) Error Program Screenshot



(g)Corrected Program Screenshot



Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

A. Lexical Errors
B. Syntax Errors
C. Semantic Errors
D. Intermediate Code Generation Issues (logic/precedence)
E. Code Optimization Issues (redundancy/inefficiency)
F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b
}

int main() {
   int #value = 20;
   float num = 15.5
   int result;

   result = divide(10);

   if(result = 5) {
      printf("Result is 5\n")
   }

   int x = 10;
   int y = 0;
   int z = x / y;

   int arr[4] = {1,2,3,4};
   printf("%d\n", arr[4]);

   int *ptr;
   *ptr = 25;

   int total;
   total = x + y * result + 0;

   while(x < 20) {
```

```
        printf("%d\n", x);
    }

    return 0;
}
```

(a) Error Identification

Line	Error	Description
4	Missing ;	The return statement is not terminated.
9	#value	# is not valid in an identifier.
10	Missing ;	The floating-point declaration needs a semicolon.
12	divide(10)	Function requires two arguments but only one is supplied.
15	result = 5	Assignment is used instead of comparison ==.
20	Missing ;	printf() statement needs a semicolon.
23	x / y	y is zero, producing division by zero at runtime.
26	arr[4]	Array has valid indexes 0 to 3; index 4 is out of bounds.
29	*ptr = 25	Pointer is uninitialized before dereferencing.
31–34	+ 0	Adding zero is redundant and unnecessary.
227–229	while(x < 20)	x is never incremented, so the loop can become infinite.

(b) Classification by Compiler Phase

1. Lexical Error

int #value = 20;

The # symbol does not form a valid identifier with value.

2. Syntax Errors

return a / b

float num = 15.5

printf("Result is 5\n")

The required semicolons are missing.

3. Semantic Error

result = divide(10);

The function divide() requires two parameters:

divide(int a, int b)

but only one argument is supplied.

4. Logical Error (Assignment Instead of Comparison)

if (result = 5) — logical error: assignment is used instead of comparison.

This performs assignment rather than comparison. It is primarily a logical error, although a compiler may issue a warning. The intended condition is:

if(result == 5)

This can produce incorrect program behaviour and affects how the expression is represented and evaluated.

5. Optimization Issue

total = x + y * result + 0;

Adding zero does not change the result:

total = x + y * result;

A compiler's optimization phase can remove such redundant operations.

6. Runtime Errors

int z = x / y;
Here y = 0.
Also:
printf("%d\n", arr[4]);
accesses an element outside the array.
Finally:
int *ptr;
*ptr = 25;
dereferences an uninitialized pointer.
These classifications follow the categories explicitly given in the assessment.

(c) Compilation vs Execution

Errors detected during compilation

The compiler can identify:

- Missing semicolons.
- Invalid identifier #value.
- Incorrect number of function arguments.
- Invalid syntax in statements.
- Assignment used in a condition may generate a warning depending on compiler settings.

Errors appearing during execution

Some problems occur when the generated program actually runs:

- Division by zero.
- Out-of-bounds array access.
- Invalid pointer dereference.
- Infinite loop.

An early syntax or lexical error can stop compilation before the compiler reaches later stages. Therefore, later semantic or runtime-related problems may not be reported until the earlier compilation errors have been corrected.

(d) Fully Corrected Program

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b;
}

int main() {
   int value = 20;
   float num = 15.5;
   int result;

   result = divide(10, 2);

   if (result == 5) {
      printf("Result is 5\n");
```

```
    }

    int x = 10;
    int y = 2;
    int z = x / y;

    int arr[4] = {1, 2, 3, 4};
    printf("%d\n", arr[3]);

    int value2 = 25;
    int *ptr = &value2;

    *ptr = 25;

    int total;
    total = x + y * result;

    while (x < 20) {
        printf("%d\n", x);
        x++;
    }

    return 0;
}
```

The corrected version maintains proper syntax, supplies the required function arguments, uses a valid identifier, avoids division by zero, accesses the array within its valid range, initializes the pointer, removes the redundant + 0, and increments x so that the loop terminates. These corrections satisfy the requirements in the assessment.

(e) Deep Analysis

1. Effect of Incorrect Pointer Usage

In the original program:

```
int *ptr;
*ptr = 25;
```

ptr is declared but does not point to a known valid memory location. Dereferencing it can cause a segmentation fault or unwanted memory modification.

A safe approach is:

```
int value2 = 25;
int *ptr = &value2;
*ptr = 25;
```

Now the pointer contains the address of a valid variable.

2. Effect of Array Bounds Violation

The array is declared as:

```
int arr[4] = {1, 2, 3, 4};
```

Therefore, the valid indexes are:

arr[0]
arr[1]
arr[2]
arr[3]
The original statement:
arr[4]
accesses memory beyond the array. This may result in an incorrect value, memory corruption, or unpredictable behaviour.

3. Operator Precedence

In:
total = x + y * result;
multiplication has higher precedence than addition. Therefore, the expression is evaluated as:
total = x + (y * result);
Incorrect grouping or precedence handling in intermediate code generation could produce a different result. The compiler therefore needs to preserve the correct order of operations while generating intermediate code.

(f) Advanced Compiler Improvements

1. Earlier Runtime Error Detection

A compiler can use **static analysis** to identify possible problems such as:

- Division by zero.
- Uninitialized pointer usage.
- Possible array-bound violations.
- Variables used before initialization.

This allows potential runtime problems to be reported before the program is executed.

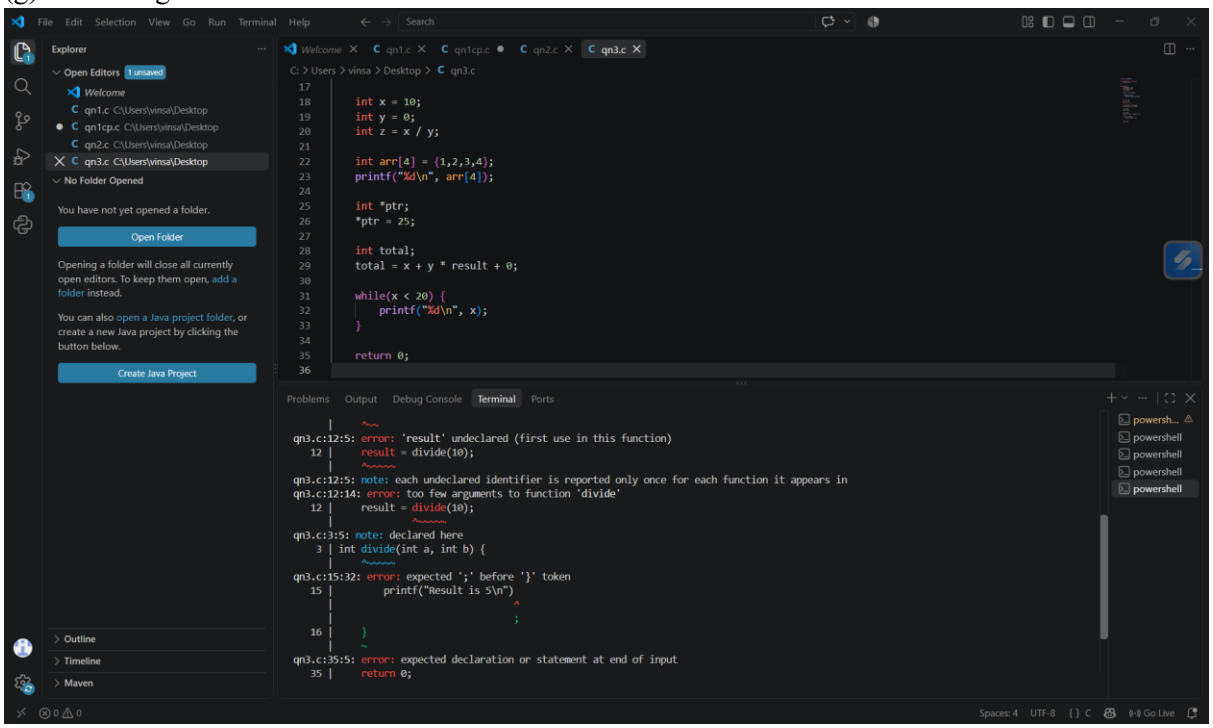
2. Better Debugging Messages

Compiler messages should contain:
Line number
↓
Problem
↓
Reason
↓
Suggested correction
For example:
Line 219: Array index 4 is outside the valid range 0–3.
Suggested correction: Use arr[3] or another valid index.

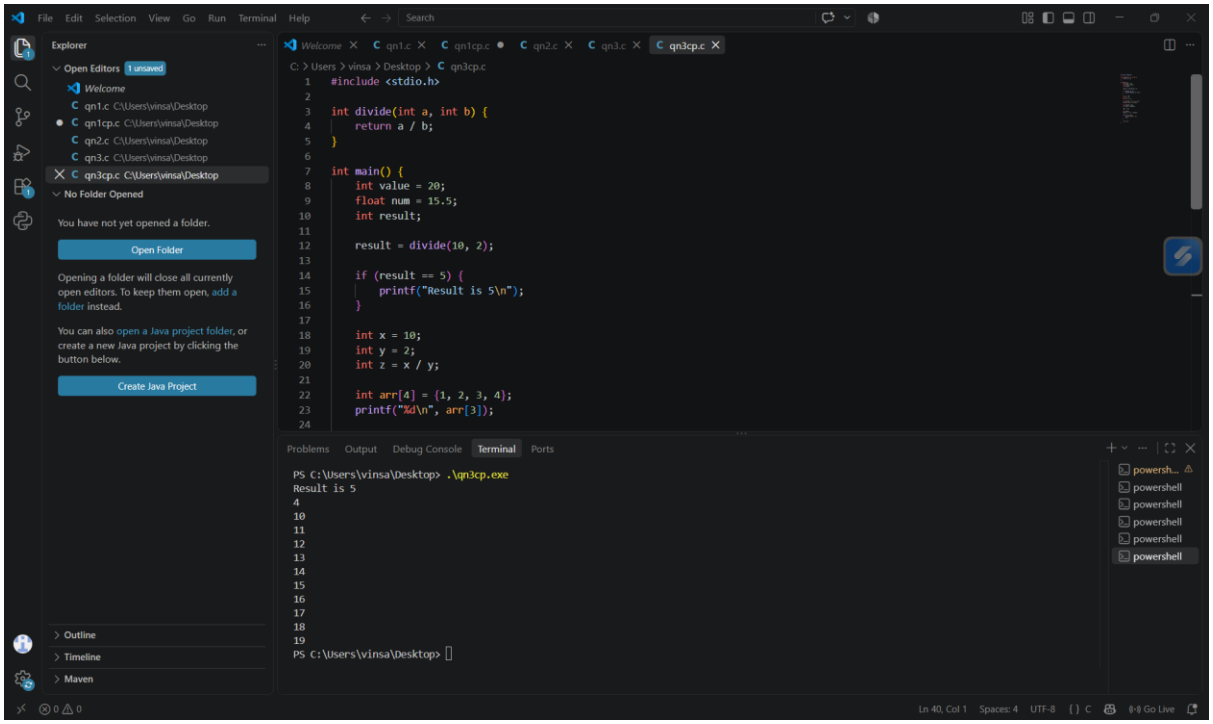
3. Optimization of Redundant Expressions

The optimization phase can identify expressions that do not change the result.
For example:
total = x + y * result + 0;
can be simplified to:
total = x + y * result;
This reduces unnecessary operations and produces more efficient intermediate/target code. The assessment specifically asks for optimization of redundant expressions.

(g) Error Program Screenshot



(h)Corrected Program Screenshot



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand